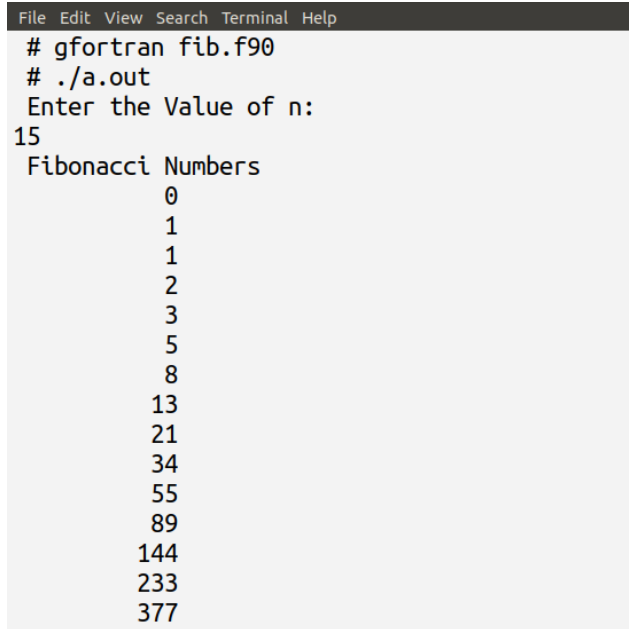


Let us execute a small program to understand the syntax and working of the Fortran programming language. Fortran programs can have a number of different extensions based on the particular version of Fortran being used. Some of the extensions include .f, .f90, .f95, .f03, .f08, etc. Fortran compilers, in general, do not require any extensions to program files. But a general suggestion I received was that it is safer to use the .f90 extension for Fortran programs denoting free-form source code. The program called *fib.f90* given below reads a number 'n' from the keyboard and prints the first 'n' Fibonacci numbers on the terminal.

```
program Fib
  integer :: f1,f2,f3,ct,n
  PRINT *, 'Enter the Value of n: '
  READ *, n
  ct = 3
  f1 = 0
  f2 = 1
  ! THIS IS A COMMENT IN FORTRAN
  PRINT *, 'Fibonacci Numbers'
  do
    f3 = f2 + f1
    if (ct <= 3) then
      if (n > 0) then
        print *, f1
      end if
      if (n > 1) then
        print *, f2
      end if
    end if
    if (n > 2) then
      print *, f3
    end if
    if (ct >= n) then
      exit
    end if
    ct = ct + 1
    f1 = f2
    f2 = f3
  end do
end program Fib
```

The program illustrates many of the important features of a Fortran program. It shows a loop statement (e.g., *do* statement), a conditional statement (e.g., *if* statement), arithmetic expressions (e.g., $f_3 = f_2 + f_1$), print statements (e.g., *print *, f1*), as well as a comment in Fortran (e.g., *! THIS IS A COMMENT IN FORTRAN*). Since printing of Fibonacci numbers is a classical



```
File Edit View Search Terminal Help
# gfortran fib.f90
# ./a.out
Enter the Value of n:
15
Fibonacci Numbers
0
1
1
2
3
5
8
13
21
34
55
89
144
233
377
```

Figure 3: Output of the program *fib.f90*

problem in computer programming, I don't think any further explanation is required. Figure 3 shows the output of the program *fib.f90*.

I must emphasise once again that the chances of young programmers coming across some Fortran code are relatively low unless they are working in the field of scientific computing. Still, there are many reasons why a programmer should at least hear the name Fortran. Let me give one more reason -- the historic significance of Fortran. If you are a professional programmer, it is essential to know about historically significant programming languages and technologies. The existence of the large amount of legacy code and current uses in scientific computing also makes Fortran a good contender as a scientific computing tool. Since Fortran is still relevant, my suggestion to young programmers is that you should at least remember the facts mentioned in this article, i.e., the Fortran standards to be used as well as how to install a compiler and execute programs in Fortran. Hence this article could also be titled 'The absolute minimum every programmer should know about Fortran'. COBOL will be taken up for discussion in the second article in this series. **END** 

By: Deepu Benson

The author is a free software enthusiast and his area of interest is theoretical computer science. The open source tools of his choice include ns-2 and ns-3. He maintains a technical blog at www.computingforbeginners.blogspot.in.